CSCE553-001-202720 Prin of Software Methodology

M2 - Assignment

M2 — Baseline Performance

CSCE 553 · Principles of Software Methodology · Fall 2026

M1 froze a working API + PostgreSQL system. You characterize the workload, predict the first limit, load the system in controlled stages, and explain the gap.

Due dates are only in Moodle.

How to run the experiments: **M2 Measurement Guide**.

Why this milestone exists

Internet-scale work starts when a **user count** is turned into a **workload**, and a workload is turned into a **capacity claim** you can falsify.

By the end of M2 you should be able to say, with numbers:

1. What “load” means for *your* product (rate, mix, shape).
2. What you predicted would break first on the M1 architecture.
3. What actually broke, at what offered load, with p50/p95/p99 — not an average.
4. Which resource moved in time with the service (CPU, memory, connections, disk, or the host falling asleep).
5. How you would revise the model for 10 thousand, 1 million, and 100 million registered users.

The architecture stays:

```
Browser / load generator  →  Your API  →  PostgreSQL
```

Back to Top

Do not change the architecture to “get better numbers.". Later milestones add cache, partitions, and so on **based on this evidence**.

What you must already have

A completed M1 baseline: public HTTPS URL, email/password login over the API, at least three demo users, documented API endpoints. You load-test **that** system.

If M1 is still incomplete, finish it first. There is nothing to measure.

What you produce

Four artifacts, all in the **same GitHub repo** as the app (plus links in Moodle):

Artifact	What it is
Workload and capacity model	Assumptions and a 10K / 1M / 100M worksheet (below)
Predicted first limit	One sentence plus the resource you expect to saturate
Measurement pack	CSVs from the runs, plus aligned resource notes/screenshots
Report	Predicted vs measured, bottleneck claim, proposed SLO, revised model

Page count is not the point. A stranger should be able to **reproduce** the main hosted and local runs from your repo.

Performance parameters (required)

Report these for every **steady-state** run. Definitions and how to collect them are in the measurement guide.

Parameter	Why it is required
Offered load	Concurrency or target RPS you set — the independent variable
Throughput	Successful responses per second

Parameter	Why it is required
Latency p50, p95, p99 (and max)	Typical vs tail; averages hide the queue
Error rate	Timeouts, 5xx, connection failures as a fraction of attempts
Resource signals	At least two of: API CPU, API memory, DB connections, DB CPU. Disk I/O if you can see it.
Cold-start latency	Hosted only: first request after idle (Render free sleep)

Your report should also state **workload mix** (read vs write fraction) and **payload** (rough response size). Hold those constant while you vary offered load.

Experiments you must run

Change **one** load dimension at a time. Stair-step, observe, **recover**, then step again. Repeat the stage nearest the knee.

Environments (both required)

Series	Where the API runs	Database	What it isolates
H — Hosted	Your M1 public URL (Render or equivalent)	Same Postgres as M1 (usually Supabase)	The system graders actually hit
L — Local API	`localhost` (same code, same commit)	Prefer the **same** hosted Postgres	The Render/host compute and sleep

Optional series **C**: local API + local Postgres. Useful if you want to see the database without the internet path. Not a substitute for H or L.

Workloads (all required)

1. **Cold start (H only).** After the service has been idle ≥ 15 minutes, one request to `GET /health`. Record wall time. Then warm up before any capacity run.
2. **Control.** Warm `GET /health` at low concurrency (1–2). This is the cheap endpoint.
3. **Read path.** Warm authenticated **list/search** (social: list messages; e-commerce: search or list products). This is your main saturation sweep.
4. **Write path.** Warm authenticated **create/update** (social: create post or like; e-commerce: add to cart or update quantity). One concurrency near the read-path knee is enough if time is short; a short sweep is better.
5. **Mix.** At one load near the knee, 80% reads / 20% writes (hold payload constant). Lecture point: mix changes the bottleneck.

Load shape (required)

For the **read path**, on **both H and L**:

- Stages: at least **1, 4, 8, 16** concurrent clients. Continue to **32** only if error rate is still low and p99 has not exploded.
- Each stage: **10 s warmup** (discard) + **60 s** measurement.
- **Stop** a series when error rate exceeds **10%**, or p99 exceeds **5 s**, or the host returns mostly 429/502/503. That stop *is* data.
- After each stage, wait until in-flight work drains (30–60 s). Do not stack stages.

Repeat the stage **just below** the stop (or the last healthy stage) **once more**. If the two repeats disagree wildly, say so; do not average them into a fake precision.

Do not pay for a larger Render instance to make the hosted curve prettier. Cold start and a 0.1 CPU free web service are part of the system under test. Name them as confounds where they distort p99; that is why series L exists.

Capacity model (required worksheet)

A registered-user count is not a workload. Fill this table **before** you look at the load-test graphs. Use the lecture identities:

- Peak RPS ≈ (active users × actions per user per hour) / 3600
- In-flight ≈ RPS × mean latency (Little's Law: $L = \lambda W$)
- Storage/day ≈ writes/day × bytes/write × copies (then add indexes/backups in words)
- Bandwidth ≈ operations/s × bytes × copies

State every assumption (active fraction, actions/hour, read/write mix, bytes per response, copies = 1 for M1).

Registered users	Active (your %)	Peak RPS	Write RPS	Storage/day	Bandwidth
10 thousand					
1 million					
100 million					

Then **predict** the first observable limit of the **current** architecture (one small API process, one Postgres). One resource, one sentence, written **before** the sweep (put a timestamp or commit in the report so it is obvious you did not reverse-engineer the prediction).

After the sweep: **revise** the 10K row with measured RPS and latency. For 1M and 100M, explain what would have to change; do not pretend the free tier will scale by hope.

Service objective (required)

Throughput alone can hide a bad service. From **your** measurements, propose **one** SLO of the form:

> 99% of **eligible** events complete within *T* ms over the lab window, with error rate ≤ *E*.

Eligible events must be defined (for example: warm `GET /api/messages` after login, excluding cold start). *T* and *E* must come from data, not from a blog. State whether series **H** meets it at the last healthy stage.

You are not operating a 28-day production SLO. You are showing you can write a **testable** threshold.

What you must not do

- Do not add a cache, queue, replica, or CDN for M2.
- Do not upgrade Render or Supabase to a paid plan to “pass” the lab.
- Do not load-test anyone else’s URL, or a production system with real users.
- Do not run unbounded tests (hours at high concurrency). Sixty seconds per stage is enough.
- Do not submit only Locust/k6 screenshots with no CSV and no resource signals.
- Do not average hosted and local numbers into one table as if they were one system.

Submission

Submit in Moodle (links, not a zip of `venv`):

1. **GitHub repository URL** (same repo as M1; add the instructor if private).
2. **Public URL** of the hosted app (unchanged from M1 unless you had to fix a bug — then say what you fixed).

Submit in Repo

3. **Report** in the repo: `m2/m2-report.md` (or PDF). Must include:
   - Worksheet (10K / 1M / 100M) and assumptions
   - Predicted first limit (pre-measurement)
   - Method: environments, endpoints, stages, stop rules, tool and version
   - Results tables for series H and L
   - Three figures: throughput vs offered load; p50 and p99 vs offered load; error rate vs offered load
   - Resource signals aligned in time with one stair-step (screenshots or notes with timestamps)
   - Cold-start number (hosted)

- Bottleneck claim: *this* resource, *because* these two signals moved together
- Proposed SLO and whether H meets it
- Gap: predicted vs measured, then the revised model
- Confounds (cold start, laptop vs cloud, shared free-tier neighbors, clock skew)

4. **Raw data** in the repo: `m2/results/` with CSVs from the driver. Do not commit secrets. Screenshots can live in `m2/results/` or the report.

How this will be graded

Criterion	What "done" looks like
Model before measure	Worksheet + dated prediction exist; assumptions are explicit
Controlled load	One factor varied; warmup/measure/recover; stop rule used
Latency tails	p50, p95, p99 reported; no "average latency" as the only number
Two environments	Hosted and local API series, not mixed into one curve
Chain of evidence	Load, service metrics, and at least two resource signals share a timeline
Bottleneck claim	One primary limit, argued from data, including mix if the write run differs
SLO	Testable threshold; eligible events defined
Reproducible	Endpoints, commands, and CSVs a teammate could rerun
No architecture cheat	Still one API + one Postgres; no paid-host "fix"

Every team member should be able to explain Little's Law on *your* numbers and where the knee is.

Getting stuck

- How to install a driver, design stages, collect CSVs, and plot: **M2 Measurement Guide**.
- Hosted first request takes ~1 minute: expected on Render free. Warm up, then measure.
- Supabase paused: resume in the dashboard, then rerun. Note it as a confound if it happened mid-lab.
- Unsure which endpoint is "the read path": use the list/search from your M1 README API table.

Last modified: Wednesday, September 2, 2026, 11:15 AM

◀ M2 - Measurement Guide

Jump to activity

M2 - Tools ▶



104 E. University Circle
Lafayette, LA 70503
(337) 482-1000

You are logged in as **Fupei Guo** (**Log out**)
Data retention summary
Get the mobile app